

①

module-1

Theory of computation

Kavya.S
Asst. professor
CSD, Dept
PESITM

July-19 Define the following terms with an example
 (i) Alphabet (ii) power of an alphabet (iii) String
 (iv) string concatenation (v) Language.

(i) Alphabet (Σ):

It is defined as a set of finite symbol

ex: $\Sigma = \{0, 1\}$ Binary alphabet

$\Sigma = \{0, 1, 2, 3, \dots, 9\}$ set for decimal numbers.

$\Sigma = \{a, b, c, d, \dots\}$ Alphabet set for english alphabetic

(ii) power of an alphabet (Σ^*)

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

ex: 1) $\Sigma = \{0, 1\}$

Σ^0 = set of string of length 0

Σ^1 = set of string of length 1

$\Sigma^* = \{0, 1, 00, 11, 10, 01, 000, 001, 111, \dots\}$

It is defined as set of strings of all possible strings length derived from Σ .

(iii) String (s):

It is defined as a finite sequence of symbols derived from alphabet (Σ)

②
ex: consider $\Sigma = \{0, 1\}$

strings that can be derived are
 $0, 1, 01, 001, 101, 1001, 101010, \dots$

(iv) String concatenation:

concatenation of two strings $(s || t)$ is defined as string appending t to s

ex: $x = ab, y = c$

$xy = abc$

(v) Language (L):

set of strings, all are chosen from Σ^* where Σ is a particular alphabet is called a language.

$$L \subseteq \Sigma^*$$

ex: consider $\Sigma = \{a, b\}$

$L_1 =$ set of string where length is odd

$L_1 = \{a, b, aaa, bbb, ababa, \dots, bbbbb\}$

$L_2 =$ set of string begin with a and end with b

$L_2 = \{ab, ababb, aabab, aabbabb, \dots\}$

(vi) Symbol: It is a building block of automata theory

ex: letter, number (0-9) etc

③

Symbols to draw an Automata:

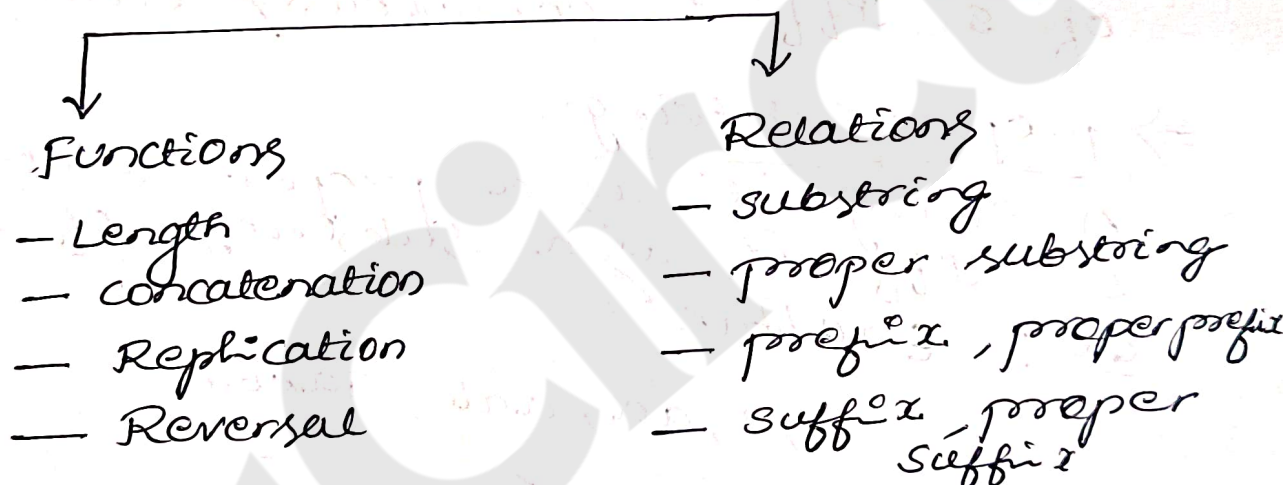
→○ → Initial state

→ → Transition

○ → Intermediate state

⊙ → End / final state

ε - epsilon: It is a special string. It is called null string or empty string.
String



1) Length: Number of symbols in the given strings

ex: $|1011| = 3$ $|\epsilon| = 0$

2) concatenation of string: concatenation of two strings $(s || t)$ is defined as string appending t to s

ex: $x = ab$, $y = c \Rightarrow xy = abc$

3) Replication: Replication of string w is defined as

$$w^0 = \epsilon$$

$$w^{i+1} = w^i \cdot w \quad \text{eg} = a^3 = aaa$$

(Repeating the occurrence of w^i no of times)

(4)

4) Replication: Replication of string 'w' is defined as $w^0 = \epsilon$

5) Reversal:

For each string w is defined as
(i) If $|w| = 0$ then $w = w^R = \epsilon$ eg: $(abc)^R = cba$

6) Relations on String

$\Rightarrow$ substring: A string 't' is a substring of s iff 't' continuously occurs in string

$\Rightarrow$ proper substring:

A string 't' is a proper substring of 's' iff it is a substring of 's' and $t \neq s$

ex: $\epsilon aabbae$

$\epsilon, a, aa, aab, aabba$

proper substring

$\Rightarrow$ prefix: A string s is a prefix of 't' iff there exists

$\exists x \in \Sigma^* (t = sx)$

$\Rightarrow$ proper prefix:

A string 's' is a proper prefix of s iff string is suffix of t and $s \neq t$

ex: consider "abba"

$\epsilon, b, bb, bba, abba$

proper suffix

5

Function on Language

All set operation like union, intersection, difference and complement can be applied.

(i) Concatenation of Language:

ex: $L_1 = \{aa, ab\}$

$$L_2 = \{xx, yy\}$$

$$L_1 L_2 = \{aaxx, aayy, abxx, abyy\}$$

Set operation on language

$$L_1 = \{\epsilon, a^2, a^4, a^6, \dots\} \text{ // even no. of a's}$$

$$L_2 = \{a^1, a^3, a^5, \dots\} \text{ // odd no. of a's}$$

$$\rightarrow L_1 \cup L_2 = \Sigma^* \text{ or } \{a\}^* \text{ // union operation}$$

$$\rightarrow L_1 \cap L_2 = \phi \text{ or } \{\epsilon\} \text{ // intersection operation}$$

$$\rightarrow L_1 - L_2 = \epsilon \text{ // difference operation}$$

$$\rightarrow \sim(L_1 - L_2) \text{ // complement operation.}$$

with neat diagram, explain a hierarchy of language classes in automata theory

$\Rightarrow$ Regular language (FSM)

$\Rightarrow$ context free language (PDA)

$\Rightarrow$ decidable language (TM)

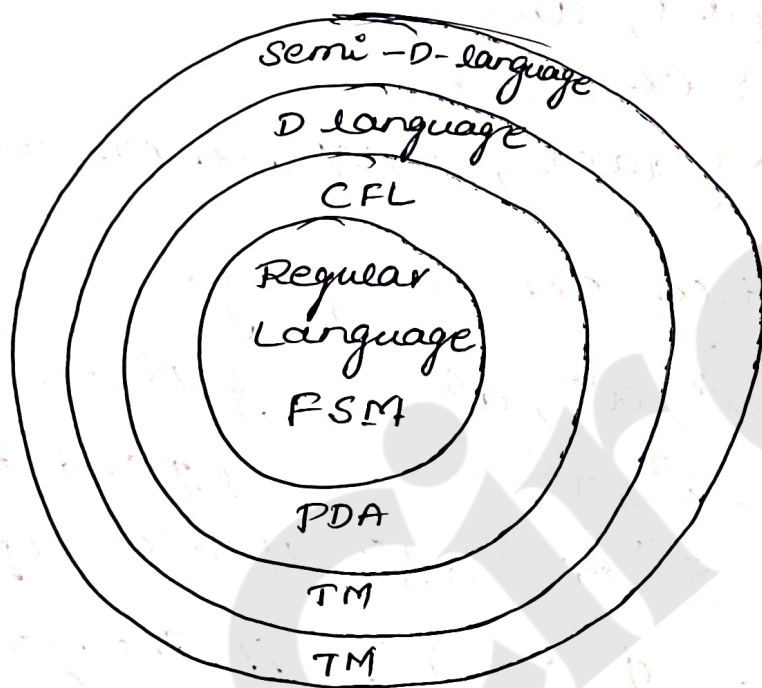
$\Rightarrow$ semidecidable language (some TM)

The tool can be selected based on
2. computational efficiency : FSM gives

6

linearly in the length of Input string
PDA grows cube of length of Input string.

TM grows exponentially with length of Input string.



2. Decidability:

FSM are decidable for example does FSM accepts some particular string

Is FSM is minimal? Are 2 FSM are identical? PDA is also decidable. But question w.r.t TM can't be answered

3. Clarity:

There are tools to design FSM and every regular language can also be described using regular expression every CFL recognized by some PDA can be described by LR grammar. No corresponding tool exists for the broader classes of decidable & semidecidable languages.

7

July 1 Define DFSM / FSM.

- * There is one and only one transition on an input
- * DFSM / FSM is formally defined as,
- * M is quintuple $(Q, \Sigma, \delta, q_0, F)$

where,

Q is finite set of state

Σ is the input alphabet

$q_0 \in Q$ is initial state

δ is the transition function

F is End / final accepting state

$$\therefore \boxed{\delta : Q \times \Sigma \rightarrow Q}$$

How to write transition diagram?

steps are,

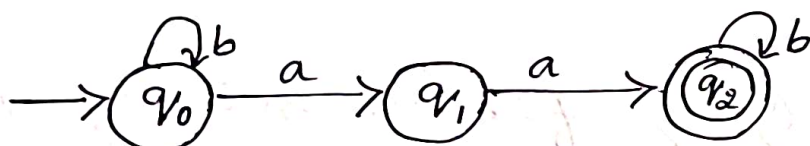
- 1) Find the minimum string accepted, this decides the number of states in the FSM, in most of the cases.
- 2) Then take longer strings and make them accepted, while modifying the transition.
- 3) Check for the minimum strings that are not to be accepted are really not accepted as per the transition diagram
- 4) See that each state has transitions equal to the number of alphabet present (Σ)

8

5) Two transition on the same alphabet do not go to different states.

problems

1) from the given transition diagram of DFsm define all the tapes.



Sol: $Q = \{q_0, q_1, q_2\}$

$M = \{Q, \Sigma, q_0, \delta, F\}$

$\Sigma = \{a, b\}$

$q_0 =$ Initial state

$F = q_2$ (final state)

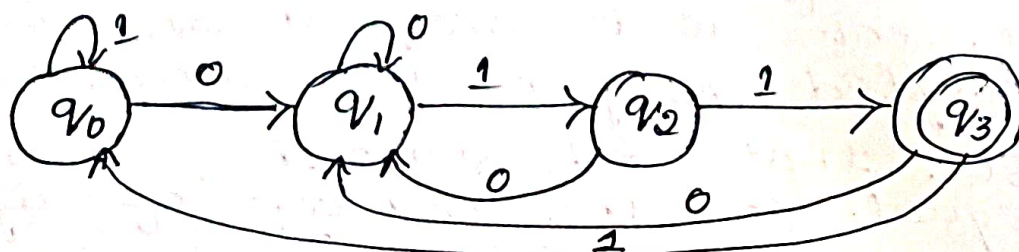
$\delta =$ is shown in transition table

δ	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_2	—
$*q_2$	—	q_2

2) Draw a DFsm to accept strings of 0's and 1's ending with string 011

Sol $\Sigma = \{0, 1\}$

$L = \{011, 0011, 1011, 10011, \dots\}$



③

3) Draw a DFA to accept string of a's & b's having substring aa

Sol $L = \{ aa, baab, abaa, aabb, \dots \}$

$$M = \{Q, \Sigma, \delta, q_0, F\}$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

q_0 = Initial state

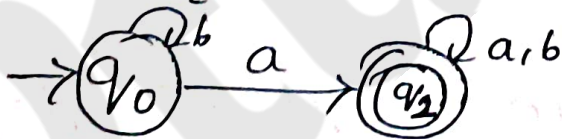
$F \rightarrow q_2$ = final state



④ Obtain a DFA to accept strings of a's and b's having atleast one a.

Sol $\Sigma = \{a, b\}$

$L = \{a, aa, aaa, abab, \dots\}$



$$Q = \{q_0, q_1\}$$

$$\Sigma = \{a, b\}$$

q_0 = Initial state

$F = q_1$ final state

10

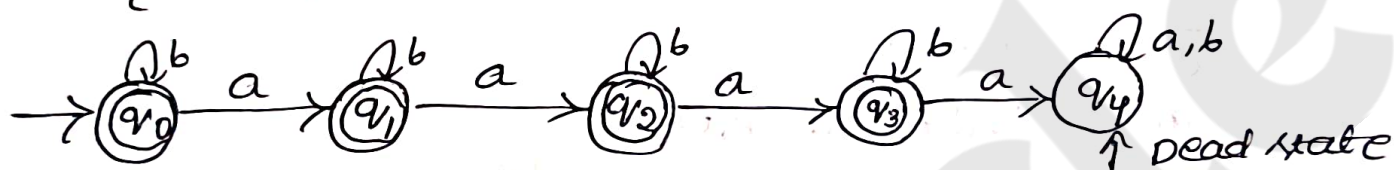
⑤ obtain a DFA to accept string a's & b's having not more than 3 a's

not more than 3 a's

can be either 0 or 1 or 2 or 3 a's

$$\Sigma = \{a, b\}$$

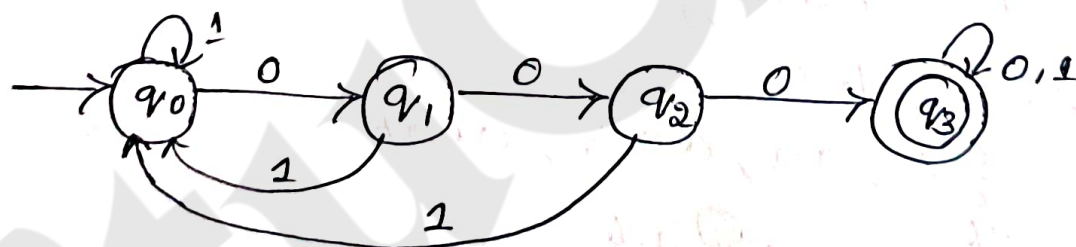
$$L = \{\epsilon, a, aa, aaa, ba, aab, \dots\}$$



⑥ Draw a DFsm that accepts strings of 0's and 1's that contain 3 consecutive 0's as a substring

$$\Sigma = \{0, 1\}$$

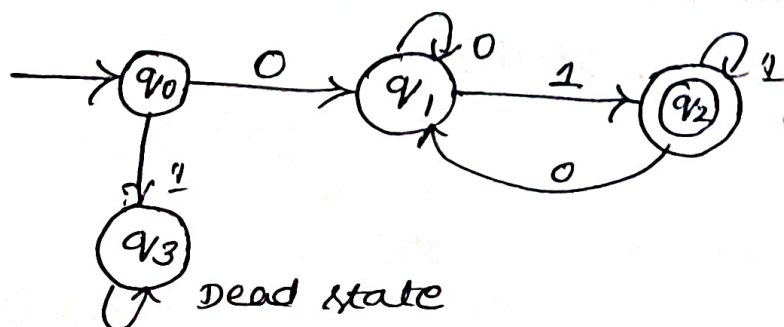
$$L = \{000, 0001, 100011, 010001100\}$$



⑦ Design a DFsm to accept strings of 0's and 1's where it begins with 0 & end with 1

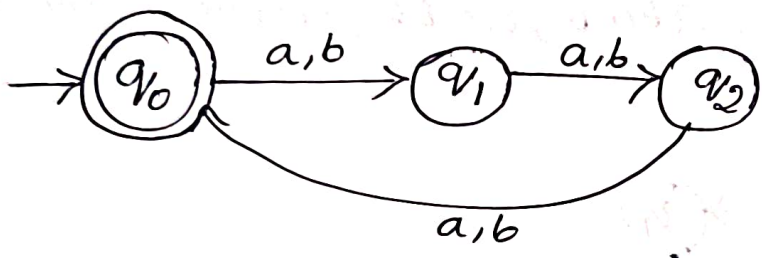
$$\Sigma = \{0, 1\}$$

$$L = \{01, 011, 01001, 011001, \dots\}$$



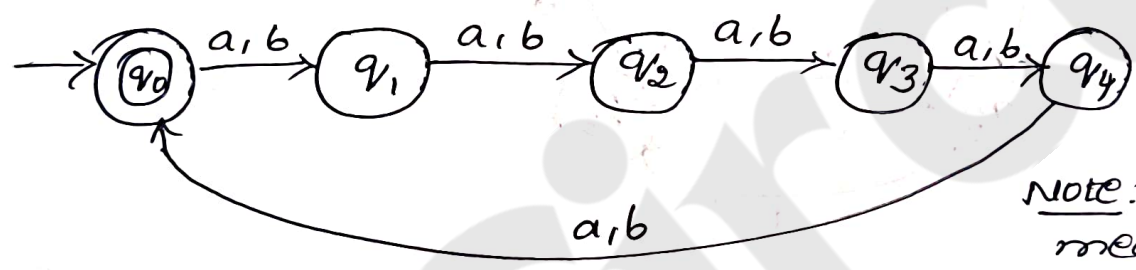
8) obtain a DFsm to accept language

(i) $L = \{w \mid |w| \bmod 3 = 0\}$ on $\Sigma = \{a, b\}$



Note: mod 3 means multiples of 3 (a,b)

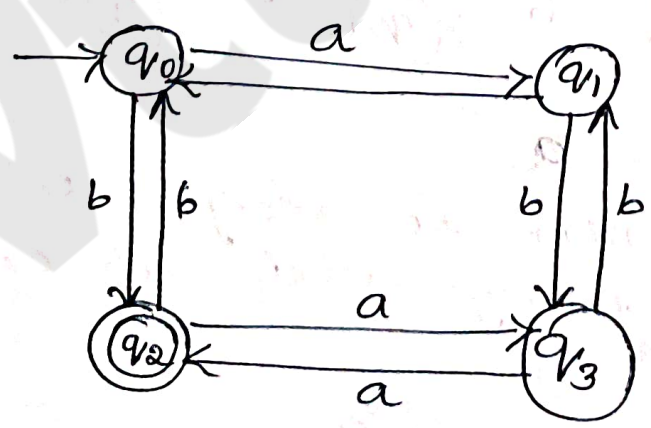
(ii) $L = \{w \mid |w| \bmod 5 = 0\}$ on $\Sigma = \{a, b\}$



Note: mod 5 means multiples of 5. (a,b)

9) $L = \{w \mid w \in \{a, b\}^* \text{ is the string with even number of a's and odd no. of b's}\}$

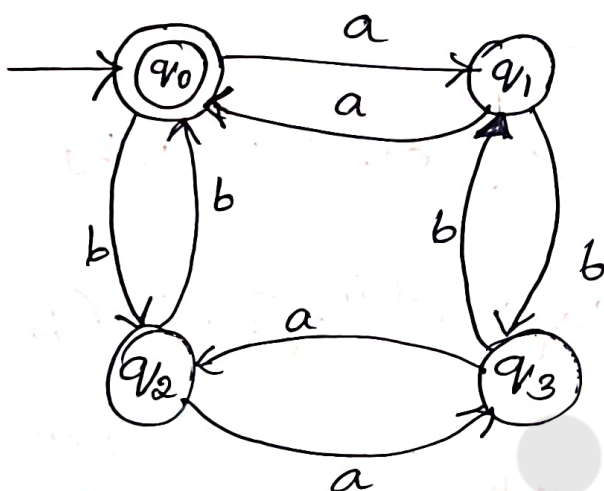
$L(m) = \{b, aab, bba, aaaaabb, \dots\}$



12

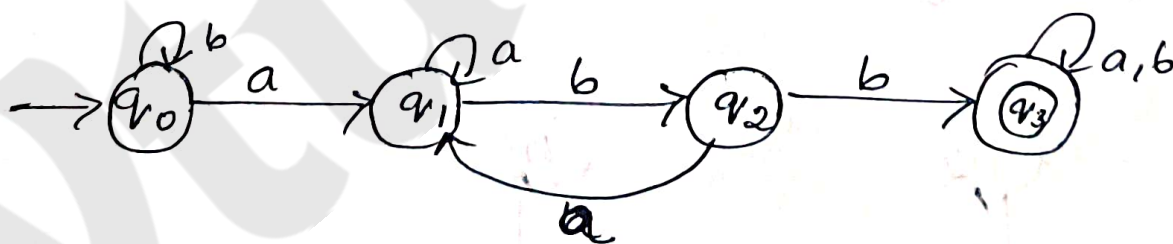
⑩ obtain a DFA to accept string of a's and b's having even number of a's and b's

$L = \{ \epsilon, aa, bb, abab, aabb, aaaa, abababab, \dots \}$

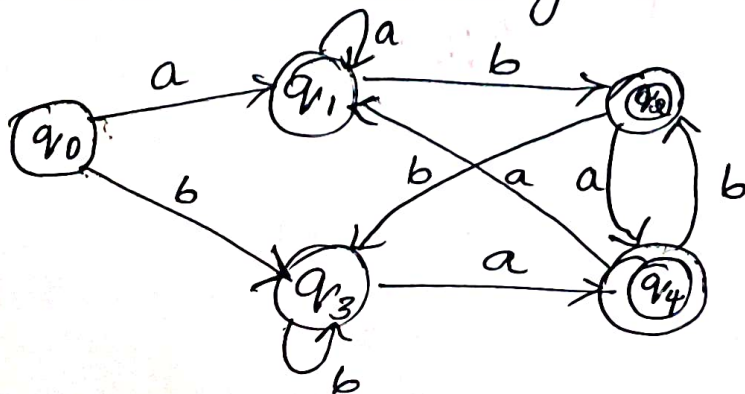


⑪ $L = \{ w \in \{a, b\}^* \mid w \text{ ends with the substring } abb \}$

$L(M) = \{ abb, aabb, baabb, \dots \}$

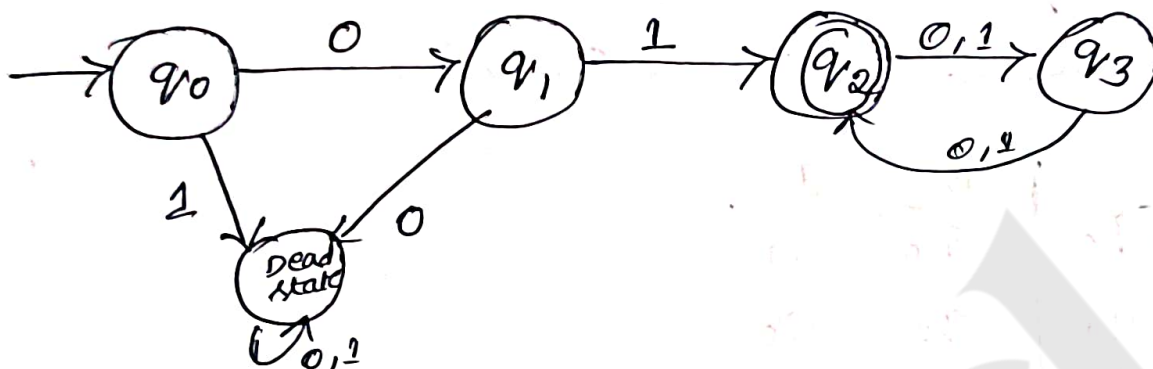


⑫ construct a DFA to accept strings of a's and b's ending with ab or ba



13

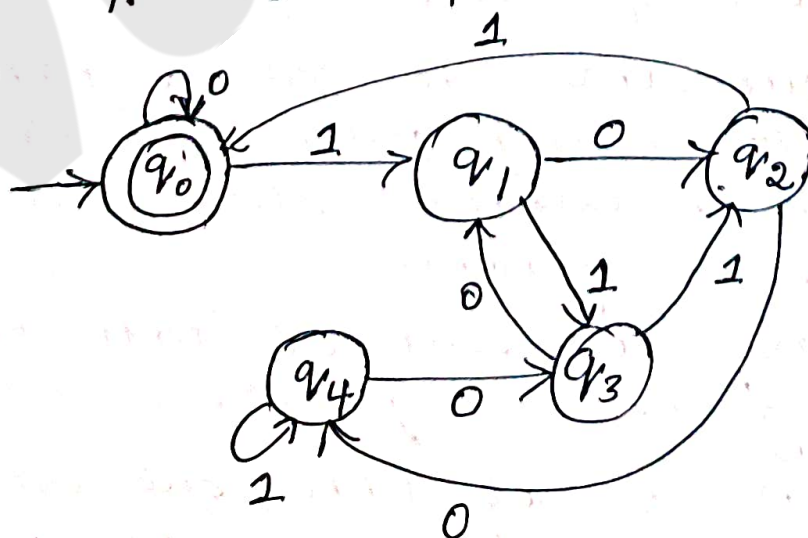
- 13) Design a DFA to accept that language
 $L = \{w \mid w \text{ is of even length \& begin with } 01\}$



- 14) Design a DFA which accept a binary number divisible by 5

sol Binary number = $\Sigma = \{0,1\}$
 divisible by 5 $\in \{0,1,2,3,4\}$

δ	0	1
$\rightarrow^* q_0$	q_0	q_1
q_1	q_2	q_3
q_2	q_4	q_0
q_3	q_1	q_2
q_4	q_3	q_4

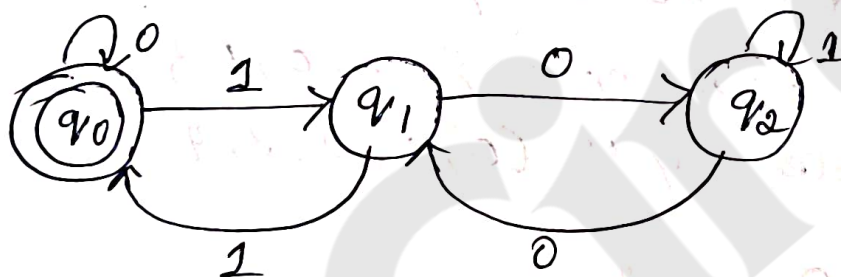


- (15) Design DFsm. to accept binary number divisible by 3

Binary number $\{0, 1\}$

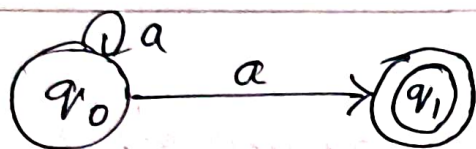
divisible by 3 $\{0, 1, 2\}$

	0	1
$\rightarrow q_0$	q_0	q_1
q_1	q_2	q_0
q_2	q_1	q_2



Non-deterministic Finite Automata (NFA)

- * In DFA there exist only one path from one state to another state on one symbol. (on specific input)
- * Finite Automata are called NFA where there exist many paths for specific input from current state to next state on a symbol.
- * It is easy to construct NFA than DFA for a given language.
- * Every NFA is not DFA, but every NFA can be translated into DFA.



possible in NFA but not in DFA

$(q_0, a) \rightarrow q_0$
 $(q_0, a) \rightarrow q_1$

2 path exist. specific path is not fixed so it is called Non-deterministic

→ showing dead state is not required in NFA.

→ NFA contains ϵ (epsilon) transition i.e. without any input, transition is possible from one state to another state

→ DFA is the subset of NFA.

Formal definition:

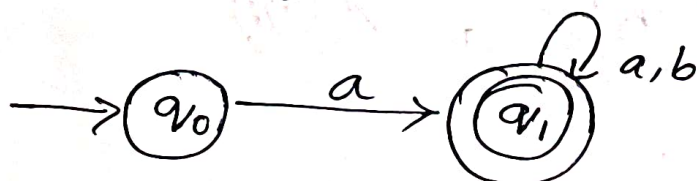
NFA having 5 states same as DFA but with different transition function.

$$M = (Q, F, \delta, q_0, \Sigma)$$

here $\delta: Q \times \Sigma \rightarrow 2^Q$

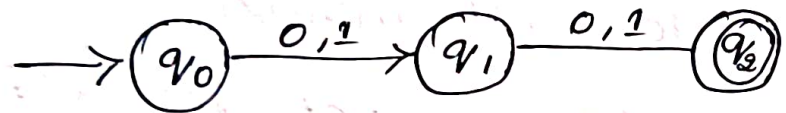
$\uparrow$ $\uparrow$ $\uparrow$
 Input state Symbol multiple output state

Ex: construct NFA for language $L = \{w \mid \text{where } w \text{ starts with } a\}$
 $\Sigma = \{a, b\}$

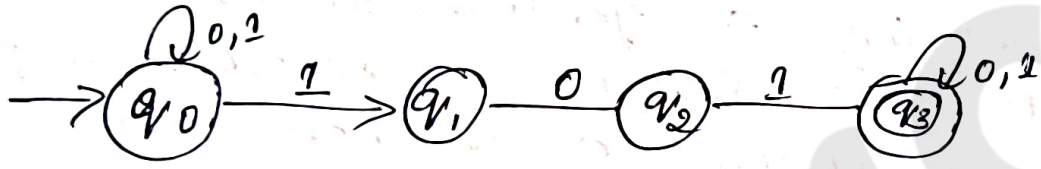


problems on NFA

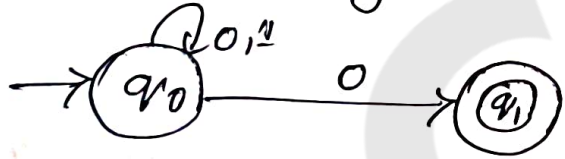
- ① construct NFA for a language L where
 $L = \{ \text{set of all strings of length 2} \}$
 $\Sigma = \{0, 1\}$, $L = \{00, 01, 10, \dots\}$



- ② Each string contains substring '101'



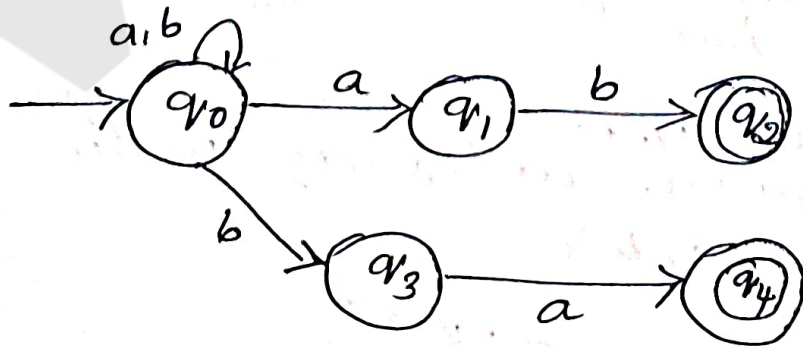
- ③ Each string that ends with '0' $\Sigma \in \{0, 1\}$



- ④ construct NFA that accepts 0's and 1's that end with 00

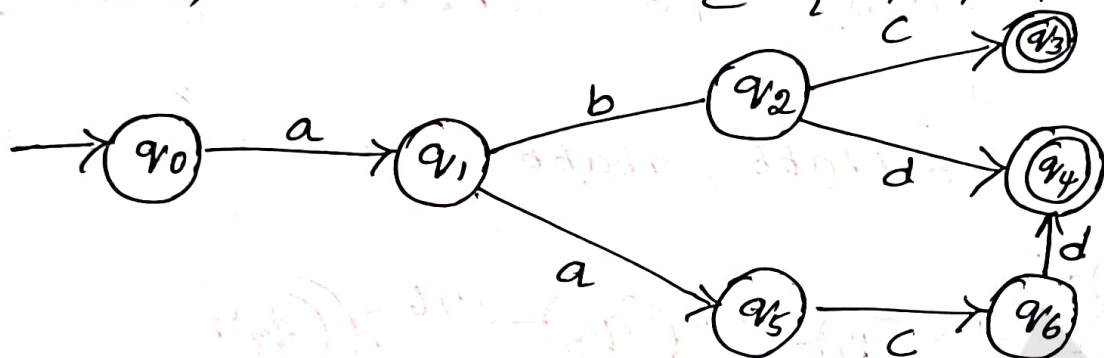


- ⑤ construct NFA to accept string ending with ab or ba over $\Sigma = \{a, b\}$



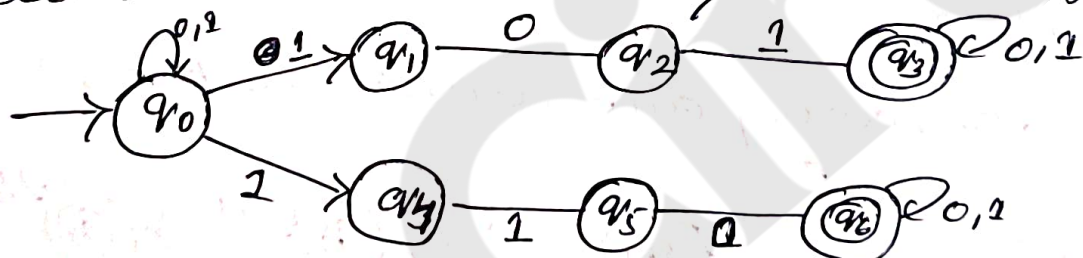
17

⑥ write NFA to recognize the string abc, abd, aacd where $\Sigma = \{a, b, c, d\}$

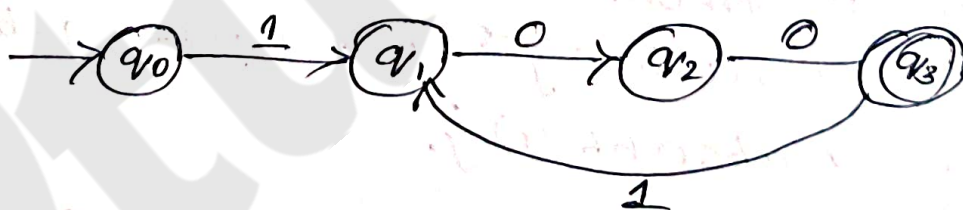


⑦ write NFA to accept the following language over $\{0, 1\}^*$

(i) set of all strings such that containing either 101 or 110 as substring



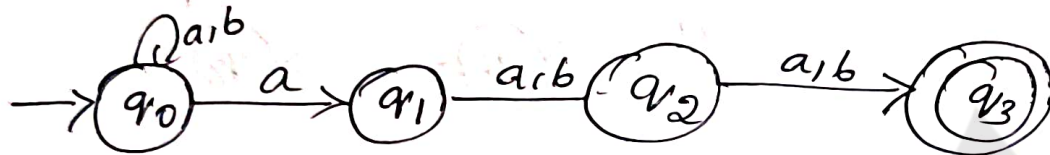
(ii) Every 1 followed by 00



⑧ construct NFA to accept language over $\{a, b\}$, containing substring ab



9) write NFSM to accept language
 $L = \{w \in \{a,b\}^* \mid 3^{rd} \text{ character from right is } a\}$
 $L = \{abb, bbbabb, ababb, aaba, aaaaa, ababa, \dots\}$

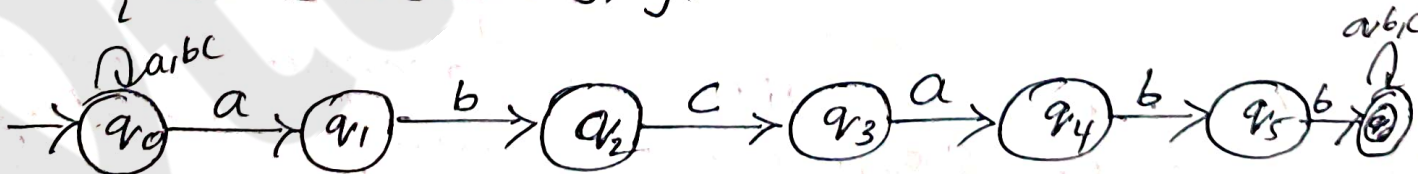


10) write NFSM to recognise the language
 $L = \{w \in \{a,b\}^* \mid w \text{ is made up of an optional } a \text{ followed by } aa, \text{ zero or more } b\text{'s}\}$

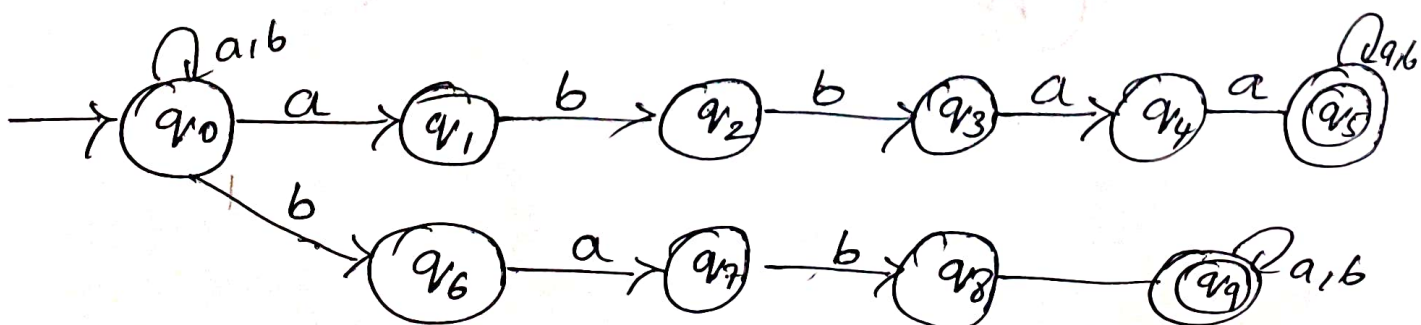
$L = \{aa, aaa, aab, aaab, aabbb, aaabbb, \dots\}$



11) construct NFSM where
 $L = \{w \in \{a,b,c\}^* : \exists x,y \in \{a,b,c\}^* \{w = xabcaabb y\}\}$



12) $L = \{w \in \{a,b\}^* : \exists x,y \in \{a,b\}^* ((w = xabbaay) \vee (w = xbabay))\}$



(19)

Finite Automata with ϵ transitions

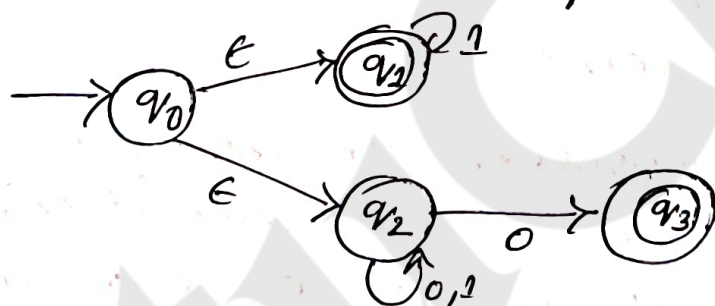
ϵ -NFA Epsilon NFA

The NFA which includes transitions on the empty input ϵ is called ϵ NFA

* ϵ -NFA is a 5 tuple or quintuple
 $M = (Q, \Sigma, \delta, q_0, F)$

* ϵ -closure of a state, is a set of all vertices p such that there is a path from q to p labelled ϵ .

ex: Draw ϵ -NFA that accept all binary string where the last symbol is 0 or that contain only 1's



$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\epsilon\text{-closure}(q_1) = \{q_1\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2, q_3\}$$

$$\epsilon\text{-closure}(q_3) = \{q_3\}$$

conversion from NFA to DFA (subset construction)

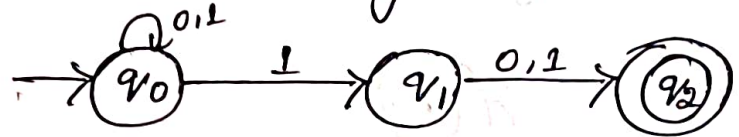
- * with an NFA, at any point in a scanning of the input string we may be faced with a choice of any number of paths to be followed to reach final state.
- * with DFA there is never a choice of paths so when we construct a ~~NFA~~ DFA which accepts the same language as a particular NFA, the conversion process effectively involve merging all possible states which can be reached on a particular input character, from a particular state, into a single, composite, state which represent all those paths.

Steps in conversion:

- Step 1: compute epsilon closure for all the state (handling ϵ transition)
- Step 2: find out ϵ -closure of old closure state (new starting state)
- Step 3: Start finding new active state
Repeat step 3 until we don't get no new state.
- Step 4: find final active states.
- Step 5: write original DFsm for NDFsm using active states.

21

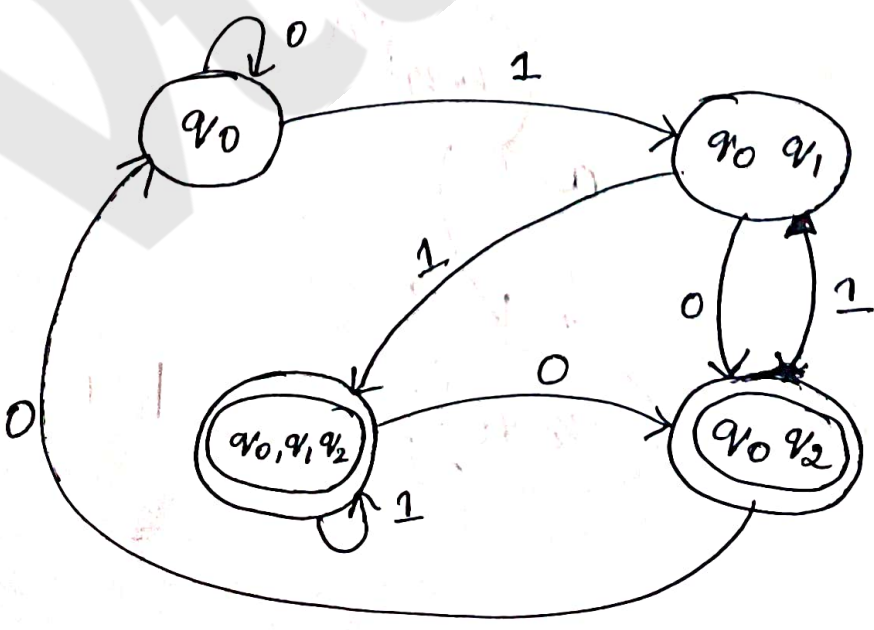
① convert the given NFA to DFA



Sol.

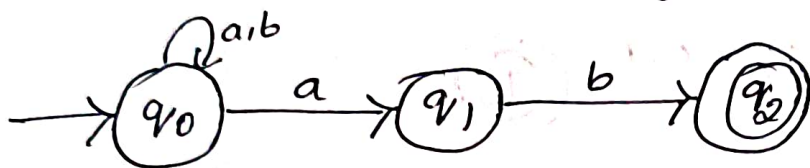
δ_{NFA}	0	1
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$
q_1	q_2	q_2
$* q_2$	$-(\phi)$	$-(\phi)$

δ_{DFA}	0	1
$\rightarrow q_0$	q_0	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$* \{q_0, q_2\}$	q_0	$\{q_0, q_1\}$
$* \{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$



Q2

convert the following NFA to DFA

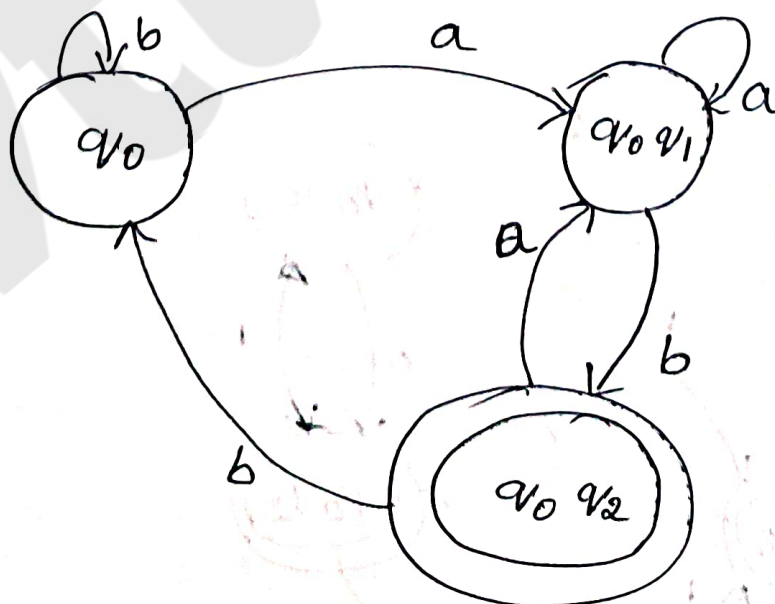


Sol

given

δ_{NFA}	a	b
$\rightarrow q_0$	$\{q_0 q_1\}$	q_0
q_1	ϕ	q_2
$*q_2$	ϕ	ϕ

δ_{DFA}	a	b
q_0	$\{q_0 q_1\}$	q_0
$\{q_0 q_1\}$	$\{q_0 q_1\}$	$\{q_0 q_2\}$
$*\{q_0 q_2\}$	$\{q_0 q_1\}$	$\{q_0\}$



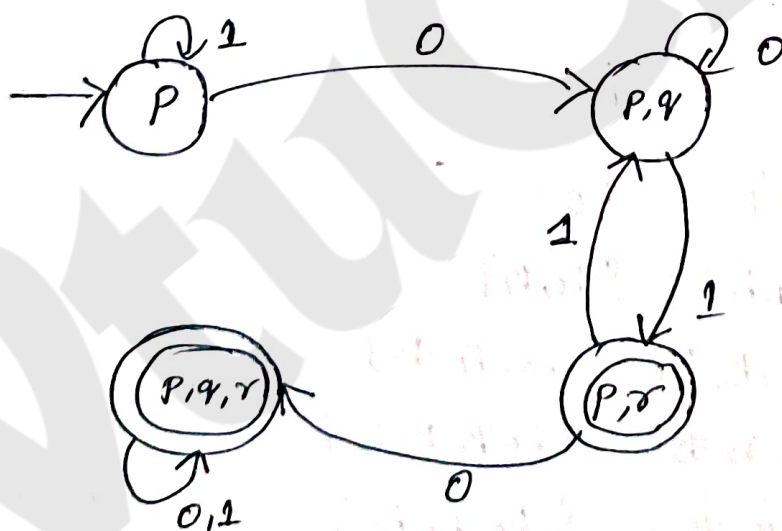
23

3) convert the following NFA to DFA

δ_{NFA}	0	1
$\rightarrow P$	$\{P, q\}$	P
q	ϕ	r
$* r$	$\{P, r\}$	q

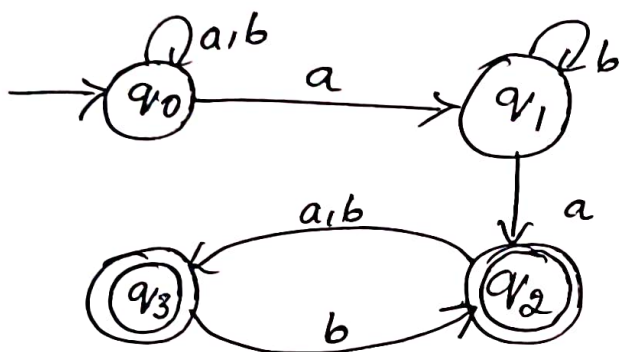
Sol

δ_{DFA}	0	1
$\rightarrow P$	$\{P, q\}$	P
$\{P, q\}$	$\{P, q\}$	$\{P, r\}$
$* \{P, r\}$	$\{P, q, r\}$	$\{P, q\}$
$* \{P, q, r\}$	$\{P, q, r\}$	$\{P, q, r\}$



24

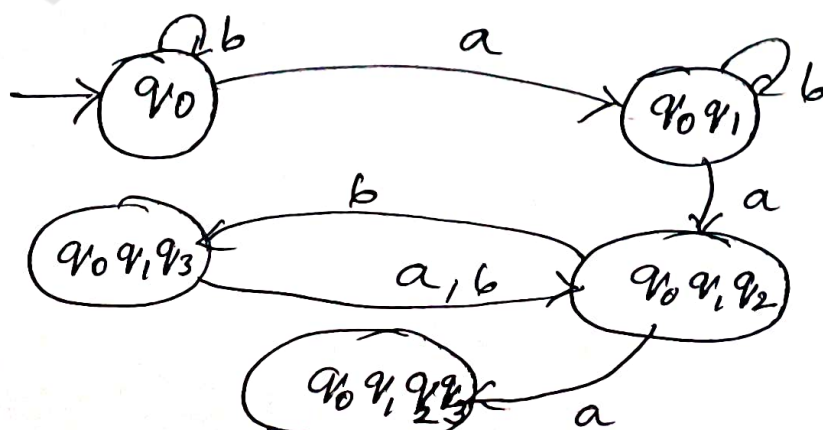
④ convert the following NFA to DFA



sol

δ_{NFA}	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	q_0
q_1	q_2	q_1
$* q_2$	q_3	q_3
$* q_3$	—	q_2

δ_{DFA}	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	q_0
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_1, q_3\}$
$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_1, q_3, q_2\}$
$\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$



5) convert the given E-NFA to DFA



E-NFA	a	b	c	ε
→ q ₀	q ₀	∅	∅	q ₁
q ₁	∅	q ₁	∅	q ₂
* q ₂	∅	∅	q ₂	∅

$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2\}$$

DFA	a	b	c
* → {q ₀ , q ₁ , q ₂ }	{q ₀ , q ₁ , q ₂ }	{q ₁ , q ₂ }	q ₂
* {q ₁ , q ₂ }	∅	{q ₁ , q ₂ }	{q ₂ }
* {q ₂ }	∅	∅	q ₂

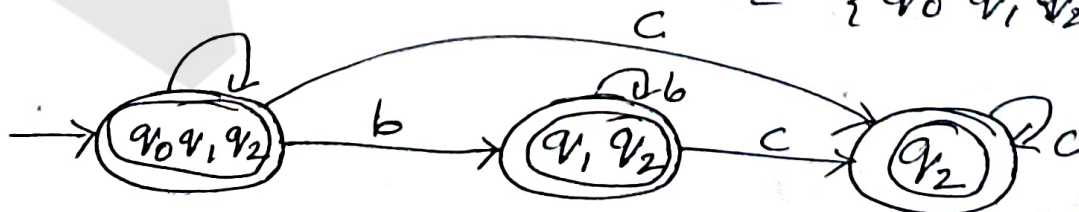
$$\delta_D(\{q_0, q_1, q_2\}, a) = \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, a))$$

$$= \epsilon\text{-closure}(\delta_{\epsilon}(q_0, a) \cup \delta_{\epsilon}(q_1, a) \cup \delta_{\epsilon}(q_2, a))$$

$$= \epsilon\text{-closure}(\{q_0\} \cup \emptyset \cup \emptyset)$$

$$= \epsilon\text{-closure}(q_0)$$

$$= \{q_0, q_1, q_2\}$$



minimization of DFA (Table filling method)

δ	0	1
$\rightarrow A$	B	C
B	D	B
* C	E	F
* D	D	B
E	A	B
F	B	C

$\{A, B, C, E, F\} \notin F$

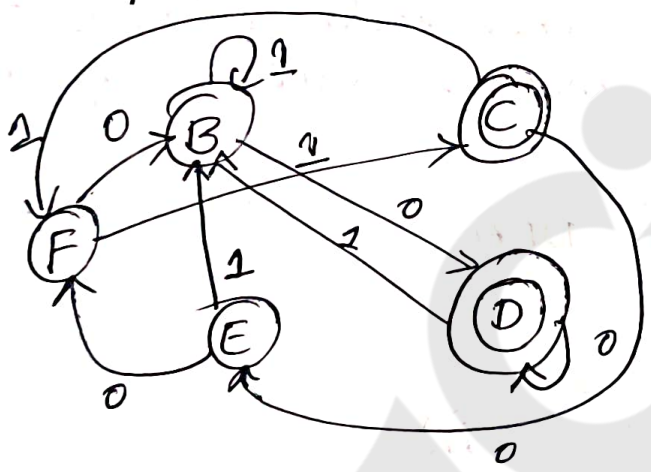
$\{C, D\} \in F$

B	(X)				
C	X	X			
D	X	X	(X)		
E	(X)	(X)	X	X	
F		(X)	X	X	(X)
	A	B	C	D	E

	0	1	
$\{A, F\}$	$\{B, B\}$	$\{C, C\}$	$= E$
$\{A, E\}$	$\{B, A\}$	$\{C, B\}$	$= NE$
$\{A, B\}$	$\{B, D\}$	$\{C, B\}$	$= NE$
$\{B, F\}$	$\{D, B\}$	$\{B, C\}$	$= NE$
$\{B, E\}$	$\{D, A\}$	$\{B, B\}$	$= NE$
$\{C, D\}$	$\{E, D\}$	$\{F, B\}$	$= NE$
$\{E, F\}$	$\{A, B\}$	$\{B, C\}$	$= NE$

A = F

δ	0	1
$\rightarrow A$	B	C
B	D	B
* C	E	F
* D	D	B
E	A	B
F	B	C



minimization of DFA means reducing the number of states from given FA

Step 1: Remove all the state that one unreachable from initial state via any set of transition of DFA

Step 2: Draw transition table for all pair of states

Step 3: Now split the transition table into two table T_1, T_2

- T_1 containing all final states
- T_2 containing non final states

Step 4 : Find similar row from T such that

$$\delta(q, a) = p$$

$$\delta(r, a) = p$$

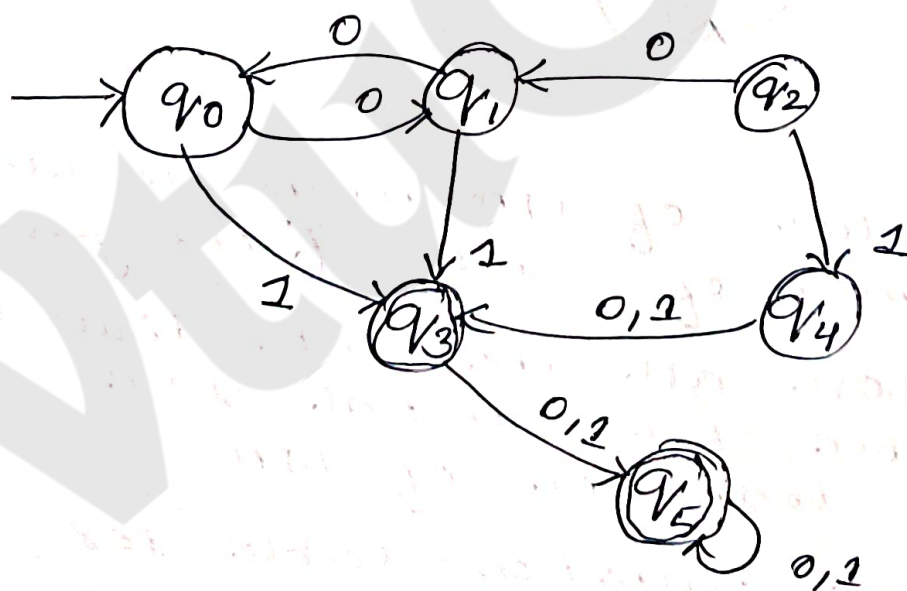
so find two states which have same value of a & b & remove one of them.

Step 5 : Repeat step 3 until we find no similar rows available in T

Step 6 : Repeat step 3 & step 4 for table T_2 also

Step 7 : Now combine the reduced T_1 & T_2 table is the final transition table of minimize DFA

② minimize given DFA



Step 1 : Remove q_2 and q_4 in finite automata

unreachable state

Step 2 Draw transition table for the rest of the state

state	0	1
$\rightarrow q_0$	q_0	q_3
q_1	q_0	q_3
$* q_3$	q_5	q_5
$* q_5$	q_5	q_5

(ii) table

Step 3: (i) Table which starts from non final state

	0	1
q_0	q_1	q_3
q_1	q_0	q_3

(ii) table which starts from final state

	0	1
$* q_3$	q_5	q_5
$* q_5$	q_5	q_5

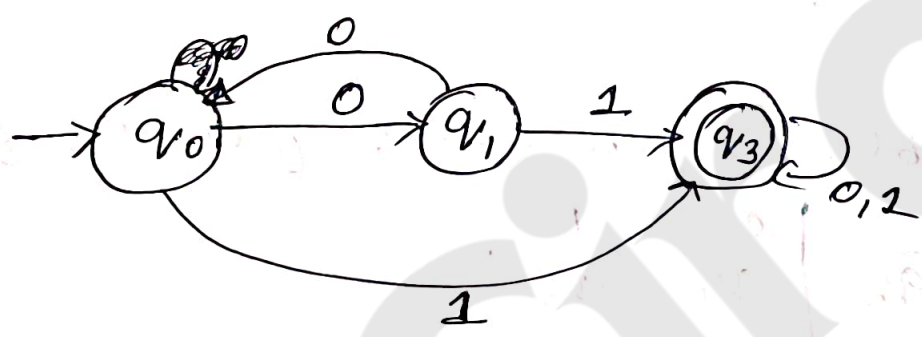
Step 4: (i) Set 1 has no similar rows, they will be same

(ii) Set 2 has similar rows, so skip q_5 & replace q_5 by q_3

	0	1
q_3	q_3	q_3

Step 6: combine set 1 & set 2

State	0	1
q_0	q_1	q_3
q_1	q_0	q_3
$*q_3$	q_3	q_3



Distinguishable and Indistinguishable

$$\delta(p, w) \& \delta(q, w) \in F$$

$$\delta(p, w) \& \delta(q, w) \notin F$$

one state move to final other state move to non final state called as distinguishable state.